mcs_demo

A set of Ultra Messaging configuration files and a test script to demonstrate setting up UM automatic monitoring using the Monitoring Collector Service (MCS). Also contains an updated version of the "lbmmon.java" example app.

Table of contents

- mcs_demo
- Table of contents
- COPYRIGHT AND LICENSE
- REPOSITORY
- INTRODUCTION
  - PREREQUISITES
  - CONFIGURATION GOALS
- DEMO ARCHITECTURE
  - HOSTS AND NETWORKS
- DEMO FILES
- RUN THE DEMO
  - SQLITE DATABASE
- OUTPUT FILES
- LBMMON.JAVA OUTPUT
- IMPORTANT STATS FIELDS
  - CONTEXT STATS
  - SOURCE STATS
    - TCP source statistics:
    - LBT-RM source statistics:
    - LBT-RU source statistics:
    - LBT-IPC source statistics:
    - LBT-SMX source statistics:
  - RECEIVER STATS
    - TCP receiver statistics:
    - LBT-RM receiver statistics:
    - LBT-RU receiver statistics:
    - LBT-IPC receiver statistics:
    - LBT-SMX receiver statistics:

COPYRIGHT AND LICENSE

All of the documentation and software included in this and any other Informatica Ultra Messaging GitHub repository Copyright (C) Informatica, 2022. All rights reserved.

Permission is granted to licensees to use or alter this software for any purpose, including commercial applications, according to the terms laid out in the Software License Agreement.

This source code example is provided by Informatica for educational and evaluation purposes only.

THE SOFTWARE IS PROVIDED "AS IS" AND INFORMATICA DISCLAIMS ALL WARRANTIES EXPRESS OR IMPLIED, INCLUDING WITHOUT LIMITATION, ANY IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. INFORMATICA DOES NOT WARRANT THAT USE OF THE SOFTWARE WILL BE UNINTERRUPTED OR ERROR-FREE. INFORMATICA SHALL NOT, UNDER ANY CIRCUMSTANCES, BE LIABLE TO LICENSEE FOR LOST PROFITS, CONSEQUENTIAL, INCIDENTAL, SPECIAL OR INDIRECT DAMAGES ARISING OUT OF OR RELATED TO THIS AGREEMENT OR THE TRANSACTIONS CONTEMPLATED HEREUNDER, EVEN IF INFORMATICA HAS BEEN APPRISED OF THE LIKELIHOOD OF SUCH DAMAGES.

REPOSITORY

See https://github.com/UltraMessaging/mcs_demo for code and documentation.

INTRODUCTION

This repository has a script and configuration files to demonstrate UM's automatic monitoring capability using the Monitoring Collector Service (MCS). The script is designed to provide all the types of monitoring data from UM services (Store, DRO, SRS) and user applications.

For more monitoring-related examples, see:

- https://github.com/UltraMessaging/mon_demo - concentrates on interpreting monitoring data
- https://github.com/UltraMessaging/mcs_json_print - user plugin to access the monitoring data in JSON instead of MCS's database.
- https://github.com/UltraMessaging/lbmmon_json - standalone program that receives the monitoring data and prints it as JSON; an alternative to running the MCS.

Informatica recommends that Ultra Messaging users enable the automatic monitoring feature in their UM-based applications and most UM daemons (Store, DRO, etc.).

This repository also contains two enhancements to existing UM exmaple applications:

- lbmmon.java - enhanced to understand SRS monitoring data.

- umercv.c - enhanced to enable the use of a UM event queue.

Note that the updates to lbmmon.java are now incorporated into the official UM release as of UM version 6.16.

Finally, there is another demo under the sub-directory json_print that uses a user-written plug-in instead of the "sqlite" database.

PREREQUISITES

You must have the following:

- Linux 64-bit system (reasonably recent).
- UMP or UMQ version 6.17, including the DRO, SRS, and MCS. The scripts hard-code versioned jar file names from UM 6.17 (for example "UMS_6.17.jar" and "protobuf-java-3.21.12.jar"). To use a later UM version, update the jar names in the "javac" and "java" commands in "tst.sh" (and in "json_print/tst.sh") to match the files in your UM package. See "$L/MCS/bin/MCS" for the jar names your MCS uses.
- A UM license key that includes persistence and the DRO (e.g. "Product=LBM,UME,UMQ,UMDRO").
- Java JDK (not just a JRE) 11 or beyond. The JDK is needed to compile "lbmmon.java". Java 8 cannot run the UM 6.17 MCS.
- gcc (to build "umercv").
- On glibc 2.34 or later (RHEL 9 and newer), the Store ("umestored") needs the "UM_Glibc_2.34" SmartHeap library fix from UM support; the library shipped with some UM 6.17 installs crashes the Store at startup with an empty log. "tst.sh" checks that the SmartHeap library first on "LD_LIBRARY_PATH" suits the host's glibc and stops with a message if it doesn't.
- sqlite3 command-line tool (reasonably recent).
- A network interface that supports multicast. The application messaging TRD "TRD1" uses multicast for both topic resolution and data (LBT-RM). Some environments (many cloud VMs, some virtual machine and WSL setups) do not pass multicast; there, the DRO will never receive data from the publisher, so the subscriber gets nothing.
- Optional: python3 (to run "peek.sh").

(Running this demo manually on Windows is reasonably straight-forward, but beyond the scope of this demo.)

CONFIGURATION GOALS

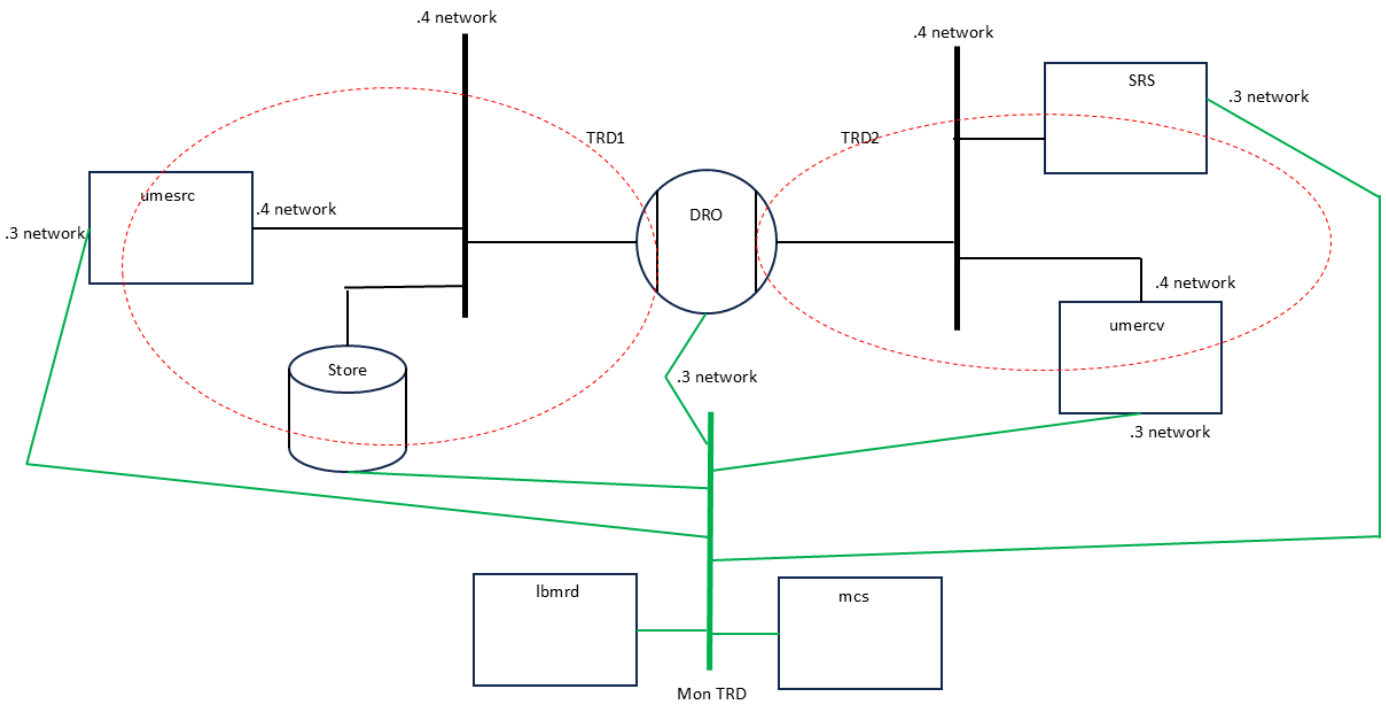
- Put monitoring data on a separate Topic Resolution Domain (TRD) from production data.
- For the monitoring TRD, use unicast UDP with the "lbmrd" for topic resolution. This is to make it easier to run on an administrative network.

- For the monitoring data, use the TCP protocol. This is to make it easier to run on an administrative network.
- Route monitoring packets to a different network interface than the production data. For example, use the administrative network. This eliminates contention for network resources. Note that in this demo, multicast is not used for monitoring.
- Disable the monitoring context's MIM and request ports. This minimizes the use of host resources.

DEMO ARCHITECTURE

The goal of this demo is to generate and collect all forms of UM monitoring data:

- Persistent Publisher (umesrc)
- Persistent Subscriber (umercv)
- Store
- DRO (Dynamic Routing Option)
- SRS (Statful Resolver Service)



The topology contains three Topic Resolution Domains (TRDs):

- TRD1 - a TRD on the .4 (10G) network whose topic resolution is implemented using multicast
- TRD2 - a TRD on the .4 (10G) network whose topic resolution is implemented using SRS.
- Mon TRD - a TRD on the .3 (1G) network whose topic resolution is implemented using lbmrd.

Of those, TRD1 and TRD2 are connected by the DRO. I.e. the subscriber "umercv" in TRD2 is able to join the persisted publisher "umesrc". The subscriber communicates with the publisher and the Store over the DRO.

The green lines represent an independent UM TRD, "Mon TRD", which is isolated from TRD1 and TRD2. Unlike TRD1 and TRD2, Mon TRD has no DRO endpoint, so it has no TRD number. Likewise, the parts of each application's shape that touch the .3 network belong to Mon TRD, even though the diagram draws no boundary around it. Each application sits in more than one TRD, which is why the shapes overlap. Mon TRD is used for monitoring data, and All components have a connection to it. The .3 monitoring network itself is physically separate from the .4 data network to ensure the monitoring data does not conflict with application data.

The MCS is also on the Mon TRD, and collects the monitoring data from the other compoents.

Note that Mon TRD does not carry any multicast traffic (monitoring data is sent using TCP).

HOSTS AND NETWORKS

The figure shows separate boxes for clarity, but "tst.sh" runs every component on a single host. That host has two network interfaces:

- 10.29.4.121 on the 10.29.4.0/24 network (the ".4" data network).
- 10.29.3.121 on the 10.29.3.0/24 network (the ".3" monitoring network).

Before running the demo, replace these addresses with your host's. The XML files use two forms of address: a network (e.g. "10.29.4.0/24"), which selects the interface, and a host address (e.g. "10.29.4.121"), which is where the SRS or lbmrd is listening. Since everything runs on one host, the host addresses are that host's own IP addresses.

File	Line content	Set to
um.xml	default_interface 10.29.4.0/24 (two places)	Data network
um.xml	resolver_service 10.29.4.121:12601	Data interface IP (SRS, TRD2)
um.xml	default_interface 10.29.3.0/24 (template "mon_ctx")	Monitoring network
um.xml	resolver_unicast_daemon ...,10.29.3.121:12801	Monitoring interface IP (lbmrd)
um.xml	resolver_multicast_address 239.101.3.1	Multicast group for TRD1 topic resolution
um.xml	transport_lbtrm_multicast_address 239.101.3.2, .4	Multicast groups for TRD1 data
srs.xml	interface 10.29.4.121	Data interface IP
srs.xml	resolver_unicast_daemon, default_interface	Same as "mon_ctx" in um.xml
lbmrd.xml	interface 10.29.3.0/24	Monitoring network
store.xml	interface 10.29.4.0/24	Data network

The multicast groups must be unused on your network, and the three groups must be different from each other.

If your host has only one network interface, use that interface's network and IP address for both the data and the monitoring entries in the table above. No other changes are needed. The monitoring and data traffic then share the interface, but the TRDs remain separate.

The demo uses these ports, which must be free on the host:

Component	Protocol	Port(s)
SRS	TCP	12601
lbmrd	UDP	12801
Store	TCP	12801
Store web monitor	TCP	12811
Monitoring contexts (lbmrd clients)	UDP	12802-12819
Application request ports	TCP	14391-14499

Other ports use UM defaults.

The "json_print" demo uses the same addresses and ports, in its own copies of the XML files.

DEMO FILES

- tst.sh - Shell script to run the demo.
- um.xml - UM library configuration file for the application messaging TRDs. This XML-format file contains UM configuration for the main messaging components (publisher, subscriber, Store, DRO).
- srs.xml - Configuration file for Stateful Resolution Service (SRS), used to provide TCP-based topic resolution for one of the application messaging TRDs.
- lbmrd.xml - Configuration file for LBM Resolver Daemon (lbmrd), used to provide unicast topic resolution for the monitoring data TRD.
- dro.xml - Configuration file for Dynamic Routing Option (DRO). The DRO routes messages between the two application messaging TRDs.
- lbm.sh.example - Model file for creating "lbm.sh" file. Provides environment and license key.
- lbmmon.java - Updated example application for collecting and printing monitoring data.
- mcs.xml - Configuration file for Monitoring Collector Service (MCS).
- mcs.properties - Additional configuration for MCS.
- store.xml - Configuration file for persistent Store.

RUN THE DEMO

1. Ensure your test system has the prerequisites.
2. Clone or download the repository at https://github.com/UltraMessaging/mcs_demo
3. Copy the file "lbm.sh.example" to "lbm.sh" and modify per your environment. I.e. insert your license key and set your file paths.
4. Edit all xml files and update IP addresses (search for "10.29"). In particular, set the multicast groups per your network in "um.xml" (search for "239.101"). See HOSTS AND NETWORKS.
5. Enter:

```
./tst.sh
```

This should take about one and a half minutes to run, and should print a series of progress messages, including PIDs of asyncronous processes.

The "mcs.out" file will contain the raw JSON records in a non-pretty format; difficult for humans to read, but easy for software tools to process. Note that the timestamps are in the form of Unix time, the number of seconds since 00:00:00 UTC on 1 January 1970, excluding leap seconds.

The "lbmmon.log" file will contain an easier to read form.

One significant difference between the two files: mcs.out's records are grouped into four sections by record type - library stats, Store stats, DRO stats, and SRS stats - with chronological ordering

within each section. In contrast, lbmmon.log's records are intermingled by type, in chronological order across record types.

SQLITE DATABASE

The sqlite database is initially created by tst.sh using the sqlite3 script contained in the UM package in the file "MCS/bin/ummon_db.sql". As of UM version 6.17, here is its content:

```
PRAGMA foreign_keys=OFF;
BEGIN TRANSACTION;
CREATE TABLE umsmonmsg(message json);
CREATE TABLE umpmonmsg(message json);
CREATE TABLE dromonmsg(message json);
CREATE TABLE srsmonmsg(message json);
COMMIT;
```

The mcs.out records are written by tst.sh using the following sqlite3 commands:

```
select * from umsmonmsg;
select * from umpmonmsg;
select * from dromonmsg;
select * from srsmonmsg;
```

OUTPUT FILES

The "output" directory has a sample of the output from running the demo in the Ultra Messaging lab.

LBMMON.JAVA OUTPUT

The "lbmmon.java" program prints statistics in human-readable form. However, if you write your own monitoring collector program, you will probably want to access the statistics individually. See displayString.md to see the field methods associated with each human-readable output line.

IMPORTANT STATS FIELDS

Most of these fields are cumulative. E.g. from one sample to the next, the value will show the total value of the stat since the object (context, source transport session, receiver transport session) was created. So, for example, if you want the topic resolution datagrams received in datagrams

per second, you'll need to take two samples and subtract, and then divide by the time difference between them.

The stats listed below are only the "most" important; there are others that might be of interest to you. For example, send_blocked or send_would_block. We recommend you browse the available stats to see what interests you.

Also, we always recommend that you record and store ALL statistics, not just the ones that interest you. It sometimes happens that, while investigating a problme, the UM Support team will ask about a stat that the user weren't interested in; if they didn't save it, Support will have trouble diagnosing their problem.

CONTEXT STATS

- tr_dgrams_sent - Get an idea of how many datagrams/sec the context is generating for topic resolution.
- tr_dgrams_rcved - Get an idea of how many datagrams/sec the entire network is generating for topic resolution.
- tr_dgrams_dropped_ver, tr_dgrams_dropped_type, tr_dgrams_dropped_malformed - If any of these are non-zero, alert the operator.
- tr_rcv_unresolved_topics - This should be zero during normal operation. Non-zero means that one or more receivers have not discovered sources.
- fragments_lost - Approximately the number of datagrams that should have been received but were lost. If this number grows by more than a few per hour, the operator should be alerted.
- fragments_unrecoverably_lost - Approximately the number of unrecoverable loss events delivered to the application. If this number is greater than zero, the operator should be alerted.

SOURCE STATS

- type - Transport type for this record.
- source - The "source string" for this transport session. This should be recorded.

A publishing application will send a separate source statistics sample for each outgoing transport session it has created. The structure of the sample depends on the transport type (TCP, LBT-RM, LBT-RU, etc).

TCP source statistics:

- num_clients - Number of receivers connected at the time the sample was taken.

LBT-RM source statistics:

- msgs_sent - Get an idea of how many datagrams/sec the transport session is generating for user messages (aggregates all topics on the transport session).

- naks_rcved - Number of individual NAKs received (not NAK packets). If this number grows by more than a few per hour, the operator should be alerted.

LBT-RU source statistics:

- msgs_sent - Get an idea of how many datagrams/sec the transport session is generating for user messages (aggregates all topics on the transport session).
- naks_rcved - Number of individual NAKs received (not NAK packets). If this number grows by more than a few per hour, the operator should be alerted.
- num_clients - Number of receivers connected at the time the sample was taken.

LBT-IPC source statistics:

- msgs_sent - Get an idea of how many datagrams/sec the transport session is generating for user messages (aggregates all topics on the transport session).
- num_clients - Number of receivers connected at the time the sample was taken.

LBT-SMX source statistics:

- msgs_sent - Get an idea of how many datagrams/sec the transport session is generating for user messages (aggregates all topics on the transport session).
- num_clients - Number of receivers connected at the time the sample was taken.

RECEIVER STATS

- type - Transport type for this record.
- source - The "source string" for this transport session. This should be recorded.

A subscribing application will send a separate receiver statistics sample for each incoming transport session it has joined. The structure of the sample depends on the transport type (TCP, LBT-RM, LBT-RU, etc).

TCP receiver statistics:

- lbm_msgs_rcved - Get an idea of how many datagrams/sec the transport session is receiving for user messages (aggregates all topics on the transport session).
- lbm_msgs_no_topic_rcved - Messages the receiver discarded because it was for a topic that is not subscribed. If this value is a significant percentage of lbm_msgs_rcved then the operator should be alerted.

LBT-RM receiver statistics:

- lbm_msgs_rcved - Get an idea of how many datagrams/sec the transport session is receiving for user messages (aggregates all topics on the transport session).

- lost - Number of datagrams that should have been received but were lost. If this number grows by more than a few per hour, the operator should be alerted.
- unrecovered_txw - Number of datagrams that were lost and could not be recovered because the source's transmission window wasn't large enough. If this number is greater than zero, the operator should be alerted.
- unrecovered_tmo - Number of datagrams that were lost and could not be recovered because it took too long. If this number is greater than zero, the operator should be alerted.
- lbm_msgs_no_topic_rcved - Messages the receiver discarded because it was for a topic that is not subscribed. If this value is a significant percentage of lbm_msgs_rcved then the operator should be alerted.
- dgrams_dropped_size, dgrams_dropped_type, dgrams_dropped_version, dgrams_dropped_hdr, dgrams_dropped_other - If any of these are non-zero, alert the operator.

LBT-RU receiver statistics:

- lbm_msgs_rcved - Get an idea of how many datagrams/sec the transport session is receiving for user messages (aggregates all topics on the transport session).
- lost - Number of datagrams that should have been received but were lost. If this number grows by more than a few per hour, the operator should be alerted.
- unrecovered_txw - Number of datagrams that were lost and could not be recovered because the source's transmission window wasn't large enough. If this number is greater than zero, the operator should be alerted.
- unrecovered_tmo - Number of datagrams that were lost and could not be recovered because it took too long. If this number is greater than zero, the operator should be alerted.
- lbm_msgs_no_topic_rcved - Messages the receiver discarded because it was for a topic that is not subscribed. If this value is a significant percentage of lbm_msgs_rcved then the operator should be alerted.
- dgrams_dropped_size, dgrams_dropped_type, dgrams_dropped_version, dgrams_dropped_hdr, dgrams_dropped_sid, dgrams_dropped_other - If any of these are non-zero, alert the operator.

LBT-IPC receiver statistics:

- lbm_msgs_rcved - Get an idea of how many datagrams/sec the transport session is receiving for user messages (aggregates all topics on the transport session).
- lbm_msgs_no_topic_rcved - Messages the receiver discarded because it was for a topic that is not subscribed. If this value is a significant percentage of lbm_msgs_rcved then the operator should be alerted.

LBT-SMX receiver statistics:

- lbm_msgs_rcved - Get an idea of how many datagrams/sec the transport session is receiving for user messages (aggregates all topics on the transport session).

- lbm_msgs_no_topic_rcved - Messages the receiver discarded because it was for a topic that is not subscribed. If this value is a significant percentage of lbm_msgs_rcved then the operator should be alerted.